

Геометрія токамака

Гайд: координати, магнітні поверхні, форма перерізу, топологія і JET — з кодом проекту

Для студентів ФМФ КПІ · 2026

Зміст

1 Як користуватися гайдом	2	4.2 Форма Міллера	13
1.1 Карта гайду	2	4.3 Навіщо плазмі форма	15
1.2 Умовні позначення і рамки	2	4.4 Як задали межу JET 1975	16
1.3 Як відтворити всі числа	3	5 Топологія: О-точка, Х-точка, сепаратриса	18
2 Координати, потік і поле	4	5.1 Критичні точки і гесіан	18
2.1 Циліндричні координати і осесиметрія	4	5.2 Магнітна вісь зблизька	18
2.2 Полоїдальний потік і поле	4	5.3 Х-точка і сепаратриса	18
2.3 Сітка EFIT і знак ψ	5	5.4 Як знаходять LCFS	19
2.4 Магнітна вісь і нормований потік	5	5.5 Скелет критичних точок як критерій якості	20
2.5 Тороїдальне поле, якого немає в даних	6	6 Геометрія JET: від плазми до котушок	22
3 Магнітні поверхні, q і кут прямих силових ліній	8	6.1 Аспектне відношення як компроміс	22
3.1 Поверхня як контур ψ	8	6.2 Котушка TF: D чистого натягу	22
3.2 Запас стійкості q	8	6.3 Гофрування тороїдального поля	23
3.3 Кут прямих силових ліній θ^*	10	6.4 Рівновага в межі JET	23
3.4 Від перерізу до тора	12	7 Практикум і задачі	27
4 Форма перерізу	13	7.1 Контрольні запитання	27
4.1 Параметри форми	13	7.2 Задачі	27
		A Відповіді і шпаргалка	28
		A.1 Відповіді до задач	28
		A.2 Шпаргалка	28
		Бібліографія	30

1. Як користуватися гайдом

Геометрія токамака — це відповідь на три запитання. *Де* лежать магнітні поверхні і як їх позначати? *Якої форми* плазма і чому саме такої? *Як з'єднано* поверхні між собою: де вісь, де Х-точка, де закінчується замкнена область? Методичка [1] відповідає на них у трьох місцях: розд. 3 (рівновага), розд. 4 (геометричні ефекти) і розд. 5 (анатомія JET). Цей гайд збирає відповіді разом і до кожної формули показує рядки коду проєкту, де її обчислюють.

Читач. Студент ФМФ 3–4 курсу, який знає Python і читав розд. 1–3 методички. Нової фізики тут майже немає. Натомість кожне означення можна перевірити, запустивши код, і кожне число в тексті має джерело: реєстр проєкту, методичку або запуск скрипту.

Два приклади. Усі числа наведено для двох машин:

- **DIII-D #203702**, кадр 75 ($t = 1760$ мс, $q_{95} = 3,70$) — реальна рівновага EFIT з набору Fusion Equilibrium Challenge [7]. Це робочий кадр 3D-візуалізації проєкту [5]: диверторна плазма з нижньою Х-точкою.
- **JET 1975** — проєкт машини [8], D-межа Міллера й аналітична рівновага Соловйова, побудована в проєкті [6]: лімітерна плазма без Х-точки.

1.1 Карта гайду

Розд.	Про що	Головний код
2	(R, φ, Z) , потік ψ , поле $\mathbf{B}$, знак ψ , нормування ψ_N , магнітна вісь	viz/src/tokviz/equilibrium.py, config.py
3	магнітні поверхні, $q(\psi)$, кут прямих силових ліній θ^* , 3D-поверхні	equilibrium.py, surfaces.py
4	параметри форми R_{geo} , a , A , κ , δ ; форма Міллера; площа й об'єм	surfaces.py, machine/components/plasma.py, geo/geom_tour.py
5	критичні точки ψ : О- і Х-точки, сепаратриса, LCFS	fusion_scoring/lcfs.py, topology_probe.py
6	JET: аспектне відношення, D-котушка, гофрування, рівновага Соловйова	machine/equilibrium_analytic.py, machine/ripple.py
7	практикум і задачі (відповіді — у дод. А)	—

1.2 Умовні позначення і рамки

Одиниці — СІ. Полоїдальний потік ψ — *на радіан* (Вб/рад), як в EFIT. φ — лише тороїдальний кут, θ — геометричний полоїдальний, θ^* — кут прямих силових ліній. Позначення ті самі, що в методичці (файл `notation.sty` спільний). Рамки:

- *Поза корпусом* — стандартна теорія, якої немає в джерелах проєкту;
- *Встановлено в проєкті [ID]* — пункт реєстру (Е — [3], VE/VR — [4]);
- *Модель проєкту* — числа нашої реконструкції JET;
- *Нове спостереження* — знайдено під час написання гайду, у реєстри ще не внесено;
- *Практикум* — що запустити і що змінити.

Код наведено *безпосередньо з файлів* проєкту: номери рядків ліворуч збігаються з файлом. Префікси шляхів: viz/ = tokamak-3d-viz/, fec/ = fusion equilibrium challenge/, ourtry/ = Our try/, geo/ = geometry/code/ (скрипт цього гайду).

1.3 Як відтворити всі числа

Практикум — один запуск

Потрібне середовище конкурсу (`numpy`, `scipy`, `scikit-image`, `pyarrow`); системний `python3` без них не підійде. З кореня проєкту:

```
PY="fusion equilibrium challenge/starter/.venv/bin/python"
```

```
"$PY" geometry/code/geom_tour.py --figs
```

Скрипт працює ≈ 6 с. Він пише всі числа гайду в `geometry/code/geom_tour.json` і перемальовує рисунки `geometry/figures/geo_*.jpg`. Дані DIII-D (`d3d_shot_203702.parquet`) уже лежать у `fusion equilibrium challenge/starter/parquet_data/`. Гайд збирається командою `cd geometry && latexmk main.tex`.

2. Координати, потік і поле

2.1 Циліндричні координати і осесиметрія

Токамак описують у циліндричних координатах (R, φ, Z) : R — відстань від вертикальної осі симетрії машини, Z — висота над екваторіальною площиною, φ — тороїдальний кут навколо осі. Площину $\varphi = \text{const}$ називають *полоїдальним перерізом*; саме в ній малюють усі «перерізи плазми». Ідеальний токамак *осесиметричний*: $\partial_\varphi = 0$ для всіх рівноважних величин. Тому вся геометрія рівноваги — двовимірна, і її повністю задає одна функція двох змінних $\psi(R, Z)$. Порушення осесиметрії (скінченне число котушок TF, розд. 6.3; резонансні збурення, розд. 3.3) розглядають як малі поправки до цієї картини.

2.2 Полоїдальний потік і поле

Поза корпусом: стандартна теорія — подання осесиметричного поля

Будь-яке осесиметричне поле з $\nabla \cdot \mathbf{B} = 0$ можна записати як

$$\mathbf{B} = \nabla\psi \times \nabla\varphi + F \nabla\varphi, \quad |\nabla\varphi| = \frac{1}{R}. \quad (2.1)$$

Підставивши $\nabla\varphi = \mathbf{e}_\varphi/R$ і $\mathbf{e}_R \times \mathbf{e}_\varphi = \mathbf{e}_Z$, $\mathbf{e}_Z \times \mathbf{e}_\varphi = -\mathbf{e}_R$, отримуємо

$$B_R = -\frac{1}{R} \frac{\partial\psi}{\partial Z}, \quad B_Z = \frac{1}{R} \frac{\partial\psi}{\partial R}, \quad B_\varphi = \frac{F}{R}, \quad B_p = \frac{|\nabla\psi|}{R}. \quad (2.2)$$

Зміст ψ : потік B_Z через горизонтальний диск радіуса R на висоті Z дорівнює $\int_0^R B_Z 2\pi R' dR' = 2\pi[\psi(R, Z) - \psi(0, Z)]$. Отже, ψ — *полоїдальний потік на радіан* тороїдального кута (звідси одиниця Вб/рад). $F = RB_\varphi$ — *полоїдальна функція струму*; у рівновазі $F = F(\psi)$ [1, розд. 3].

Найважливіший наслідок (2.1): $\mathbf{B} \cdot \nabla\psi = 0$. Силова лінія ніколи не перетинає поверхню $\psi = \text{const}$, тож кожна лінія рівня ψ у перерізі, обернена навколо осі Z , дає *магнітну поверхню* — вкладений тор. Уся геометрія рівноваги — це геометрія ліній рівня однієї функції.

У коді формули (2.2) записано дослівно. Функцію ψ задано на сітці EFIT і інтерпольовано бікубічним сплайном, похідні — аналітичні похідні сплайна:

viz/src/tokviz/equilibrium.py, рядки 176–189 (поле з ψ і F)

```
176 # -- axisymmetric field -----
177 def B_R(self, r, z):
178     return -self.dpsi_dZ(r, z) / r
179
180 def B_Z(self, r, z):
181     return self.dpsi_dR(r, z) / r
182
183 def B_pol(self, r, z):
184     return self.grad_psi_mag(r, z) / r
185
186 def B_phi(self, r, z=None):
187     if self._F is None:
188         raise RuntimeError("F not set -- call calibrate_F() or set_F() first")
189     return self._F / np.asarray(r, dtype=float)
```

Рядки 177–184 — три компоненти (2.2); dpsi_dR , dpsi_dZ (рядки 167–171 файлу) — похідні сплайна. Рядки 186–189: $B_\varphi = F/R$ зі *сталим* F . Для DIII-D це наближення: у реальній плазмі F залежить від ψ (діа- і парамагнітний ефект струмів плазми), але в наборі даних $F(\psi)$ немає (п. 2.5). Рівновага Соловйова для JET (розд. 6) використовує самоузгоджену $F(\psi)$.

2.3 Сітка EFIT і знак ψ

У наборі Fusion Equilibrium Challenge кожен кадр — масив ψ розміром 65×65 (рядки — Z , стовпці — R) з кроком $\Delta R = 26,6$ мм, $\Delta Z = 50,0$ мм для DIII-D (значення з запуску `geom_tour.py`). Сітка груба: на мале коло радіусом 10 см навколо осі припадає лише кілька вузлів. Тому все, що потребує других похідних (кривина поверхонь біля осі, гесіан у X-точці, розд. 5), треба читати з поправкою на цю роздільність.

Знак ψ не універсальний. Формула (2.1) фіксує лише, що поле визначає ψ з точністю до сталої; напрямки струму плазми і поля TF різні на різних машинах, і EFIT кожної машини має власну конвенцію. У DIII-D магнітна вісь — *мінімум* ψ , у MAST — максимум. Проєкт зводить обидва випадки до одного множником:

`viz/src/tokviz/config.py`, рядки 39–43 (конвенція знака)

```
39 #: EFIT sign convention per machine. DIII-D stores the magnetic axis at the
40 #: MINIMUM of psi, MAST at the maximum. Multiplying psi by this makes the axis
41 #: a maximum in every case, which is what the 0-point finder assumes.
42 #: (Matches ``fusion_scoring/common.py:AXIS_SIGN`` in the challenge scorer.)
43 AXIS_SIGN = {"DIII-D": -1.0, "MAST": +1.0}
```

Після множення на `AXIS_SIGN` вісь завжди максимум, і всі пошукові алгоритми (вісь, LCFS) пишуть один раз. Той самий словник є в оцінювачі конкурсу (`fusion_scoring/common.py`). Забутий знак — не дрібниця: у проєкті збої вилючення LCFS при перенесенні DIII-D→MAST, які спершу прийняли за фізичний сигнал (гіпотеза C8), виявилися саме артефактом знака ψ [1, розд. 4, R9].

2.4 Магнітна вісь і нормований потік

Магнітна вісь — точка, де $\nabla\psi = 0$ і ψ досягає екстремуму (О-точка, розд. 5). Пошук іде у два кроки: грубий максимум `axis_sign ·  $\psi$` на внутрішніх вузлах сітки, потім уточнення методом Нелдера–Міда на сплайні:

`viz/src/tokviz/equilibrium.py`, рядки 201–225 (пошук магнітної осі)

```
201 def find_axis(self) -> tuple[float, float, float]:
202     """Locate the 0-point by maximising ``axis_sign * psi``.
203
204     Coarse grid argmax, then Nelder-Mead on the spline for sub-cell
205     accuracy. Returns ``(R_axis, Z_axis, psi_axis)``.
206     """
207     phi = self.axis_sign * self.psi
208     # restrict to the interior so an edge cell cannot win
209     interior = np.full(phi.shape, -np.inf)
210     interior[2:-2, 2:-2] = phi[2:-2, 2:-2]
211     iz, ir = np.unravel_index(np.argmax(interior), phi.shape)
212     r0, z0 = float(self.R[ir]), float(self.Z[iz])
213
214     def neg(p):
215         r, z = p
216         if not (self.R[0] <= r <= self.R[-1] and self.Z[0] <= z <= self.Z[-1]):
217             return 1e9
218         return -float(self.axis_sign * self._spl.ev(r, z))
219
220     res = minimize(neg, np.array([r0, z0]), method="Nelder-Mead",
221                   options=dict(xatol=1e-7, fatol=1e-12, maxiter=2000))
222     r, z = float(res.x[0]), float(res.x[1])
223     self.r_axis, self.z_axis = r, z
224     self.psi_axis = float(self._spl.ev(r, z))
225     return r, z, self.psi_axis
```

Рядки 209–211: крайні два ряди вузлів виключено, щоб максимум на межі сітки (поле котушок) не «виграв» у справжньої осі. Рядки 214–218: цільова функція $-\text{axis_sign} \cdot \psi$ на сплайні, зі штрафом за вихід із сітки. Рядки 220–225: Нелдер–Мід з допуском 10^{-7} м.

Встановлено в проєкті [VE1]

Знайдена вісь збігається з віссю EFIT до $\Delta R = 0,0002$ мм, $\Delta Z = 0,0001$ мм при комірці $26,6 \times 50,0$ мм [4]. Наш запуск дає $R_{ax} = 1,67817$ м, $Z_{ax} = 0,02623$ м проти EFIT $1,67817$ м і $0,02623$ м.

Далі потрібна межа. Замість того щоб шукати її самостійно, код бере ψ_b як медіану ψ уздовж контуру LCFS, який EFIT надає разом з рівновагою. Тоді нормування збігається з EFIT-івським:

viz/src/tokviz/equilibrium.py, рядки 227–247 (межа і нормований потік)

```
227 def set_boundary_from_contour(self, lcfs_r, lcfs_z) -> float:
228     """Take psi_bdy as the median psi along a supplied LCFS polygon.
229
230     Using the dataset's own ``efit_lcfs_r/z`` keeps our normalisation
231     identical to EFIT's rather than re-deriving a separatrix.
232     """
233     vals = self.psi_at(np.asarray(lcfs_r, float), np.asarray(lcfs_z, float))
234     self.psi_bdy = float(np.median(vals))
235     return self.psi_bdy
236
237 def psi_n(self, r, z):
238     """Normalised flux: 0 on axis, 1 on the boundary."""
239     if self.psi_axis is None or self.psi_bdy is None:
240         raise RuntimeError("call find_axis() and set_boundary_from_contour() first")
241     return (self.psi_at(r, z) - self.psi_axis) / (self.psi_bdy - self.psi_axis)
242
243 def psi_n_grid(self) -> np.ndarray:
244     return (self.psi - self.psi_axis) / (self.psi_bdy - self.psi_axis)
245
246 def psi_of_psi_n(self, psi_n) -> np.ndarray:
247     return self.psi_axis + np.asarray(psi_n, float) * (self.psi_bdy - self.psi_axis)
```

Рядки 237–241 — означення

$$\psi_N = \frac{\psi - \psi_{ax}}{\psi_b - \psi_{ax}}, \quad \psi_N = 0 \text{ на осі, } \psi_N = 1 \text{ на межі.} \quad (2.3)$$

ψ_N не залежить ні від знака, ні від масштабу ψ . Саме тому нею позначають поверхні в усіх графіках і таблицях: «поверхня $\psi_N = 0,95$ » означає одне й те саме для DIII-D і MAST. Для кадру 75: $\psi_{ax} = -0,2932$ Вб/рад, $\psi_b = 0,0752$ Вб/рад; ψ зростає назовні, бо вісь DIII-D — мінімум.

Встановлено в проєкті [VE2]

Контур LCFS з набору даних справді є лінією рівня ψ : розкид ψ уздовж нього становить $8,2 \cdot 10^{-5}$, тобто 0,022 % діапазону потоку. Це виправдовує медіану як ψ_b [4].

2.5 Тороїдальне поле, якого немає в даних

ψ дає лише полоїдальне поле. Для B_φ потрібна F , а набір Fusion Equilibrium Challenge її не містить. Проєкт відновлює F з того, що в наборі є, — q_{95} . Оскільки q лінійне за F (розд. 3.2), F знаходять одним діленням; деталі — в п. 3.2.

Встановлено в проєкті [VE3]

$F = 3,23341$ Т м дає $B_\varphi(R_{ax}) = 1,927$ Тл, тоді як реальне поле DIII-D становить 1,9–2,1 Тл. Жодної інформації про B_0 у розрахунок не подавали [4]. На геометричному центрі $R_{geo} = 1,656$ м маємо $B_\varphi = 1,953$ Тл.

Корисне відчуття масштабу: на зовнішньому екваторі межі ($R \approx 2,26$ м) $B_p = 0,359$ Тл, а $B_\varphi = F/R \approx 1,43$ Тл. Тобто силова лінія нахилена до тороїдального напрямку лише на $\arctan(0,359/1,43) \approx 14^\circ$: вона робить кілька обертів навколо головної осі, поки обійде переріз один раз. Скільки саме — каже q .

Практикум — поле і знак

(1) У `geom_tour.py` замініть `machine="DIII-D"` на `"MAST"` для того самого кадру DIII-D. Що знайде `find_axis()`? Чому? (2) Візьміть `MAST mast_shot_28348.parquet`: чи збігається знайдена вісь з `efit_r_axis`, `efit_z_axis`? (3) Порахуйте B_p і B_φ на внутрішньому екваторі ($R \approx 1,05$ м) і кут нахилу лінії. Де лінія крутіша — зовні чи всередині?

3. Магнітні поверхні, q і кут прямих силових ліній

3.1 Поверхня як контур ψ

Магнітна поверхня в перерізі — замкнена лінія рівня $\psi_N = s$, що охоплює вісь. Лінія рівня на дискретній сітці — це стандартна задача «marching squares»; проєкт бере її з `scikit-image` і відсіює все зайве:

viz/src/tokviz/equilibrium.py, рядки 250–280 (поверхня $\psi_N = s$)

```
250 def surface(self, psi_n_level: float) -> np.ndarray | None:
251     """Closed flux-surface polygon at a normalised flux level.
252
253     Returns an ``(N, 2)`` array of ``(R, Z)`` , oriented counter-clockwise,
254     with the first point NOT repeated -- or ``None`` if no closed contour
255     encircling the axis exists at that level.
256     """
257     from skimage import measure
258
259     level = float(self.psi_of_psi_n(psi_n_level))
260     cans = measure.find_contours(self.psi, level)
261     best = None
262     for c in cans:
263         # c is (row, col) = (Z index, R index), fractional
264         rr = self.R[0] + c[:, 1] * self.dR
265         zz = self.Z[0] + c[:, 0] * self.dZ
266         closed = (abs(rr[0] - rr[-1]) < 1e-9) and (abs(zz[0] - zz[-1]) < 1e-9)
267         if not closed:
268             continue
269         pts = np.column_stack([rr[:-1], zz[:-1]])
270         if pts.shape[0] < 16:
271             continue
272         if not _encircles(pts, self.r_axis, self.z_axis):
273             continue
274         if best is None or pts.shape[0] > best.shape[0]:
275             best = pts
276     if best is None:
277         return None
278     if _signed_area(best) < 0:
279         best = best[::-1]
280     return best
```

Рядок 260: усі контури рівня s . На одному рівні їх може бути кілька: крім поверхні навколо осі, трапляються контури в приватній області під Х-точкою або біля котушок. Рядки 263–265: дробові індекси сітки переводяться в метри. Рядки 266–273 відкидають незамкнені контури (вони виходять на межу сітки), короткі (< 16 точок) і ті, що не охоплюють вісь (`_encircles`: кут навколо осі набігає більш ніж на $1,5\pi$). Рядки 278–279 орієнтують контур проти годинникової стрілки за знаком площі. Орієнтація потрібна, щоб полоїдальний кут зростав у передбачуваному напрямку.

Під Х-точкою (розд. 5) замкненого контуру навколо осі при $\psi_N \geq 1$ немає. Тому `surface(1.0)` на диверторній рівновазі ненадійна, і проєкт працює з $\psi_N \leq 0,98\text{--}0,99$. Звідси й звичай характеризувати край плазми величиною $q_{95} = q(\psi_N = 0,95)$, а не q на самій межі: на сепаратрисі $q \rightarrow \infty$ (п. 5.3).

3.2 Запас стійкості q

q — скільки разів силова лінія обходить тор уздовж φ за один полоїдальний оберт. Уздовж лінії $R d\varphi/d\ell = B_\varphi/B_p$, де $d\ell$ — елемент полоїдальної дуги. За один полоїдальний оберт з (2.2)

$$\Delta\varphi = \oint \frac{B_\varphi}{R B_p} d\ell = F \oint \frac{d\ell}{R |\nabla\psi|}, \quad q = \frac{\Delta\varphi}{2\pi} = \frac{F}{2\pi} \oint \frac{d\ell}{R |\nabla\psi|}. \quad (3.1)$$

Це та сама формула, що й $q = \frac{1}{2\pi} \oint F |\mathcal{J}| R^{-2} d\theta$ у методичці [1, розд. 4, (4.1)]: для кута, що вимірює довжину дуги, якобіан $|\mathcal{J}| = R d\ell / (|\nabla\psi| d\theta)$. Інтеграл береться чисельно, сумою по відрізках контуру:

viz/src/tokviz/equilibrium.py, рядки 283–306 (контурний інтеграл i q)

```

283 def _loop_integral(self, pts: np.ndarray) -> tuple[np.ndarray, float]:
284     """Cumulative and total of ``dl / (R |grad psi|)`` around a surface.
285
286     ``q = (F / 2pi) * total`` and ``theta* = 2pi * cumulative / total``,
287     so the straight-field-line angle is independent of F.
288     """
289     r, z = pts[:, 0], pts[:, 1]
290     rc = np.r_[r, r[0]]
291     zc = np.r_[z, z[0]]
292     dl = np.hypot(np.diff(rc), np.diff(zc)) # (N,)
293     rm = 0.5 * (rc[:-1] + rc[1:])
294     zm = 0.5 * (zc[:-1] + zc[1:])
295     g = self.grad_psi_mag(rm, zm)
296     g = np.where(g < 1e-12, 1e-12, g)
297     w = dl / (rm * g)
298     cum = np.concatenate([[0.0], np.cumsum(w)]) # (N+1,)
299     return cum, float(cum[-1])
300
301 def q_at(self, psi_n_level: float) -> float | None:
302     pts = self.surface(psi_n_level)
303     if pts is None:
304         return None
305     _, tot = self._loop_integral(pts)
306     return float(self.F * tot / TWO_PI)

```

Рядки 290–292: довжини відрізків $d\ell$ замкненого багатокутника. Рядки 293–297: вага $d\ell / (R|\nabla\psi|)$ у середині кожного відрізка; рядок 296 захищає від ділення на нуль біля Х-точки, де $|\nabla\psi| \rightarrow 0$. Рядок 298: накопичена сума cum — вона ще знадобиться для θ^* . Рядок 306: q за (3.1).

Формула (3.1) лінійна за F , тому F відновлюють з відомого q_{95} одним діленням (п. 2.5): viz/src/tokviz/equilibrium.py, рядки 308–322 (калібрування F за q_{95})

```

308 def calibrate_F(self, q95_target: float, psi_n: float = 0.95) -> float:
309     """Solve ``F`` so the computed q at ``psi_n`` equals the dataset's q95.
310
311     The challenge dataset withholds ``fpol``, so B_phi cannot be derived
312     from psi alone. Since q is linear in F, this is a single division --
313     no iteration, no assumed vacuum field.
314     """
315     pts = self.surface(psi_n)
316     if pts is None:
317         raise RuntimeError(f"no closed surface at psi_n={psi_n}")
318     _, tot = self._loop_integral(pts)
319     if tot <= 0:
320         raise RuntimeError("degenerate loop integral")
321     self._F = float(TWO_PI * abs(q95_target) / tot)
322     return self._F

```

ψ_N	0,10	0,30	0,50	0,70	0,90	0,95	0,98
q (DIII-D #203702, кадр 75)	0,759	0,956	1,262	1,791	2,999	3,699*	4,497

* — за побудовою (калібрування F). Запуск geom_tour.py.

Профіль типовий для диверторної плазми: q повільно зростає в ядрі й круто — біля межі, де поверхні проходять поруч з Х-точкою і $|\nabla\psi|$ малий. У центрі $q < 1$: поверхня $q = 1$ лежить біля $\psi_N \approx 0,33$ (саме там у ранній версії візуалізації з'явився фальшивий острів, VE6 нижче).

Встановлено в проєкті [VE4, VE16]

Два незалежні способи дають те саме q : контурний інтеграл (3.1) і пряме трасування силової лінії (тороїдальні оберти на полоїдальні) збігаються до 0,001–0,16% на $\psi_N =$

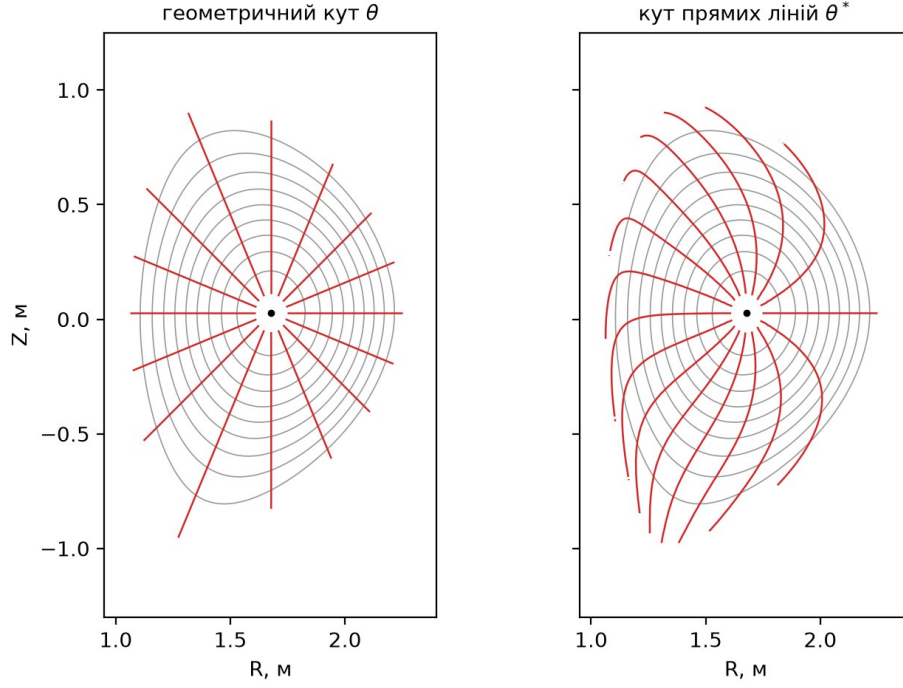


Рис. 3.1. Лінії сталою кута (червоні, крок $\pi/8$) на поверхнях $\psi_N = 0,1 \dots 0,9$ DIII-D #203702, кадр 75. Ліворуч — геометричний кут θ (промені з осі), праворуч — кут прямих силових ліній θ^* , відлічений від зовнішнього екватора. Лінії θ^* густі на внутрішньому боці: там силова лінія проводить більше тороїдальної «часу» на одиницю пологої дуги. Власний розрахунок, код: `geometry/code/geom_tour.py`.

0,3...0,95 (VE4). Число обертання $1/q$, виміряне трасуванням, відтворює рівноважне до 0,03 % (VE16) [4].

3.3 Кут прямих силових ліній θ^*

У геометричному куті $\theta = \arctan[(Z - Z_{ax})/(R - R_{ax})]$ силова лінія на поверхні — крива: на внутрішньому боці тора вона йде пологої повільніше, ніж на зовнішньому. Кут прямих силових ліній θ^* вибирають так, щоб лінія просувалася рівномірно:

$$\frac{d\theta^*}{d\varphi} = \frac{1}{q} \quad \Rightarrow \quad \theta^*(\ell) = 2\pi \frac{\int_0^\ell d\ell'/(R|\nabla\psi|)}{\oint d\ell'/(R|\nabla\psi|)}. \quad (3.2)$$

Друга рівність випливає з першої: з (3.1) на відрізку $d\ell$ лінія проходить $d\varphi = F d\ell/(R|\nabla\psi|)$, а $d\theta^* = d\varphi/q$. Отже, θ^* — це просто нормована накопичена сума sum з контурного інтеграла. F скорочується, тому θ^* від F не залежить.

Навіщо стільки клопоту? Збурення вигляду $\cos(m\theta^* - n\varphi)$ на поверхні, де $q = m/n$, має сталу фазу вздовж силової лінії і тому *резонує*: саме там виростає магнітний острів m/n . У геометричному куті така гармоніка «розмазується» по кількох m і дає острови не там (VR1 нижче). На рис. 3.1 видно, наскільки кути різні: найбільше відхилення $|\theta^* - \theta|$ становить 15° на $\psi_N = 0,2$, 25° на $0,5$ і 48° на $0,9$ (з наближенням до X-точки воно зростає).

viz/src/tokviz/equilibrium.py, рядки 363–395 (таблиця $\Delta = \theta^* - \theta$)

```

363     for s_lvl in levels:
364         pts = self.surface(float(s_lvl))
365         if pts is None:
366             continue
367         cum, tot = self._loop_integral(pts)
368         if tot <= 0:
369             continue
370         th_star = TWO_PI * cum[:-1] / tot                    # (N,), 0 -> 2pi
371         tg = np.arctan2(pts[:, 1] - self.z_axis, pts[:, 0] - self.r_axis)
372         tgu = np.unwrap(tg)
373
374         # One strictly monotonic poloidal circuit. Fails near an X-point,
375         # where the surface is no longer star-shaped about the axis.
376         if np.any(np.diff(tgu) <= 0):
377             continue
378         gain = tgu[-1] - tgu[0]
379         if not (0.75 * TWO_PI < gain < TWO_PI):
380             continue
381
382         # Delta = theta* - theta_geo is single-valued and 2pi-periodic,
383         # because both angles gain exactly 2pi per circuit.
384         delta = th_star - tgu
385         a = np.mod(tgu, TWO_PI)
386         order = np.argsort(a)
387         a_s, d_s = a[order], delta[order]
388         a_ext = np.r_[a_s[-1] - TWO_PI, a_s, a_s[0] + TWO_PI]
389         d_ext = np.r_[d_s[-1], d_s, d_s[0]]
390         keep_d.append(np.interp(th_uniform, a_ext, d_ext))
391         keep_s.append(float(s_lvl))
392
393     if not keep_s:
394         raise RuntimeError("no usable flux surfaces for theta* construction")
395     return np.asarray(keep_s), th_uniform, np.asarray(keep_d)

```

Код табулює не сам θ^* , а різницю $\Delta = \theta^* - \theta$ на регулярній сітці (ψ_N, θ) . Рядок 370: θ^* з накопиченої суми (3.2). Рядки 371–372: геометричний кут кожної точки контуру, «розгорнутий» без стрибків 2π . Рядки 376–380 пропускають поверхні, де θ уздовж контуру не монотонний (біля Х-точки поверхня перестає бути «зіркоподібною» відносно осі). Рядки 384–390: Δ однозначна і 2π -періодична, бо обидва кути набігають рівно 2π за оберт; її інтерполюють на рівномірну сітку θ . Похідні θ^* по R і Z далі беруть за правилом ланцюга (клас `ThetaStarField`, рядки 410–481 файлу).

Встановлено в проєкті [VE6, VR1, VR2]

Перша реалізація растеризувала $\cos(m\theta^*)$ на сітку 65×65 . Це дало 2,4 % паразитної гармоніки $m = 1$ і фальшивий ланцюг островів на $q = 1$ ($\psi_N = 0,335$), хоча збурення містило лише $m = 2$ і 3. Аналітичне обчислення через табульовану Δ зменшило паразитну $m = 1$ до 0,4 %, і фальшивий острів зник (VE6). Звідси два спростування: «геометричного кута достатньо для розміщення островів» (VR1) і «паразитні гармоніки з інтерполяції можна ігнорувати» (VR2) [4].

Нове спостереження (не внесено в реєстри) — початок відліку θ^*

θ^* визначено з точністю до сталої на кожній поверхні. У `theta_star_geometry` ця стала — кут першої точки контуру, яку повертає `find_contours` (рядок 370: `cum` починається з нуля там, де почався контур). Ця точка на сусідніх поверхнях різна. Запуск `geom_tour.py` на кадрі 75 дає: $\Delta(\theta = 0)$ по 200 поверхнях таблиці лежить у межах $-79,7^\circ \dots -36,6^\circ$, а між сусідніми поверхнями ($\delta\psi_N \approx 0,005$) стрибає до $34,6^\circ$.

Для θ^* на одній поверхні це неважливо, але `ThetaStarField.value_and_grad` бере ще й $\partial\Delta/\partial\psi_N$ (рядки 479–480 файлу), і стрибки потрапляють у градієнт θ^* , а через нього — у поле збурення (`perturbation.py`, рядки 156–157). Середньоквадратичне $|\nabla\theta^*|$ уздовж поверхні, рад/м:

ψ_N	0,3	0,6	0,9
як у кодї	38,0	3,9	3,03
з відліком від зовнішнього екватора	3,5	2,5	2,89

На $\psi_N = 0,3$ градієнт завищено в 11 разів, на $\psi_N = 0,9$, де лежать робочі резонанси проєкту ($q = 3, 10/3, 11/3$), — на 5 %. Це узгоджується з тим, що виміряні ширини островів збігаються з формулою (VE18). *Вплив на реєстр VE/VR не перевірено.* Можливе виправлення в один рядок: після рядка 390 віднімати від кожного рядка таблиці Δ його значення при $\theta = 0$. Так зроблено в `geom_tour.py`, рядки 104–109, і на рис. 3.1.

3.4 Від перерізу до тора

Осесиметрія робить 3D-геометрію тривіальною: будь-який контур (R, Z) — межа плазми, поверхня ψ_N , профіль котушки — обертають навколо осі Z . Точка (R, Z) на куті φ переходить у $(x, y, z) = (R \cos \varphi, R \sin \varphi, Z)$:

`viz/src/tokviz/surfaces.py`, рядки 38–68 (поверхня обертання)

```

38 def revolve(profile_rz: np.ndarray, n_tor: int = 128, scale: float = 1.0,
39             phi0: float = 0.0, phi1: float = TWO_PI, closed_tor: bool = True):
40     """Sweep a closed (R, Z) profile around the Z axis.
41
42     Returns ``(verts (n_pol*n_tor, 3), faces (n_pol*n_seg, 4))``.
43     """
44     prof = np.asarray(profile_rz, dtype=float) * float(scale)
45     n_pol = prof.shape[0]
46     if closed_tor:
47        phis = np.linspace(phi0, phi1, n_tor, endpoint=False)
48     else:
49        phis = np.linspace(phi0, phi1, n_tor)
50     c, s = np.cos(phis), np.sin(phis)
51
52     X = prof[:, 0][None, :] * c[:, None]      # (n_tor, n_pol)
53     Y = prof[:, 0][None, :] * s[:, None]
54     Z = np.repeat(prof[:, 1][None, :], phis.size, axis=0)
55     verts = np.stack([X, Y, Z], axis=-1).reshape(-1, 3)
56
57     n_seg = n_tor if closed_tor else n_tor - 1
58     i = np.arange(n_seg)[:, None]
59     j = np.arange(n_pol)[None, :]
60     i1 = (i + 1) % n_tor
61     j1 = (j + 1) % n_pol
62     faces = np.stack([
63         (i * n_pol + j).ravel(),
64         (i1 * n_pol + j).ravel(),
65         (i1 * n_pol + j1).ravel(),
66         (i * n_pol + j1).ravel(),
67     ], axis=-1)
68     return verts, faces.astype(np.int32)

```

Рядки 52–55 — саме ця формула для всіх точок профілю й усіх кутів одразу. Рядки 57–67 збирають чотирикутні грані між сусідніми кільцями; `% n_tor` і `% n_pol` замикають сітку по обох кутах. Параметр `scale` множить R і Z однаково. Для рендера проєкт збільшує DPI-D у 3 рази (`GEOMETRY_SCALE` у `config.py`): аспектне відношення при цьому не змінюється, а отже й форма поверхонь та розташування островів у нормованих координатах.

Практикум — q і θ^*

(1) Порахуйте q на $\psi_N = 0,5$ контурним інтегралом при `surface()` на сітці, згущеній удвічі (інтерполуйте ψ сплайном на 129×129 і збудуйте нову `Equilibrium`). Наскільки зміниться q ? (2) Знайдіть ψ_N поверхні $q = 1$ бісекцією по `q_at`. (3) Повторіть таблицю «Нового спостереження» для кадрів 10 і 150 того самого розряду. Чи змінюються стрибки початку відліку?

4. Форма перерізу

4.1 Параметри форми

Форму межі плазми (LCFS) описують кількома числами, які читають за її крайніми точками. Позначимо $R_{\max}$, $R_{\min}$ — крайні радіуси, $Z_{\max}$, $Z_{\min}$ — крайні висоти, а $R_{Z_{\max}}$, $R_{Z_{\min}}$ — радіуси найвищої й найнижчої точок.

Поза корпусом: стандартна теорія — означення [1], розд. 4

$$\begin{aligned} R_{\text{geo}} &= \frac{R_{\max} + R_{\min}}{2}, & a &= \frac{R_{\max} - R_{\min}}{2}, & A &= \frac{R_{\text{geo}}}{a}, & \epsilon &= \frac{1}{A}, \\ \kappa &= \frac{Z_{\max} - Z_{\min}}{2a}, & \delta_u &= \frac{R_{\text{geo}} - R_{Z_{\max}}}{a}, & \delta_l &= \frac{R_{\text{geo}} - R_{Z_{\min}}}{a}, & \delta &= \frac{\delta_u + \delta_l}{2}. \end{aligned} \quad (4.1)$$

A — аспектне відношення, ϵ — обернене аспектне відношення, κ — витягнутість (elongation), δ — трикутність (triangularity). $\delta > 0$: верхівка зсунута *всередину* від R_{geo} (класичний D); $\delta < 0$ — назовні («перевернутий D»).

У коді це кілька рядків:

geo/geom_tour.py, рядки 37–58 (параметри форми, площа, об'єм)

```
37 def shape_params(R, Z):
38     """R_geo, a, A, kappa, delta_u, delta_l, delta за крайніми точками контуру."""
39     R, Z = np.asarray(R, float), np.asarray(Z, float)
40     Rmax, Rmin = R.max(), R.min()
41     R_geo = 0.5 * (Rmax + Rmin)
42     a = 0.5 * (Rmax - Rmin)
43     iu, il = np.argmax(Z), np.argmin(Z)
44     kappa = (Z[iu] - Z[il]) / (2 * a)
45     d_u = (R_geo - R[iu]) / a # вершня трикутність
46     d_l = (R_geo - R[il]) / a # нижня трикутність
47     return {"R_geo": R_geo, "a": a, "A": R_geo / a, "kappa": kappa,
48           "delta_u": d_u, "delta_l": d_l, "delta": 0.5 * (d_u + d_l),
49           "Z_geo": 0.5 * (Z[iu] + Z[il]), "R_top": R[iu], "Z_top": Z[iu],
50           "R_bot": R[il], "Z_bot": Z[il]}
51
52
53 def area_volume(R, Z):
54     """Площа перерізу (формула Гаусса) і об'єм тора (теорема Паппа--Гульдіна)."""
55     R, Z = np.asarray(R, float), np.asarray(Z, float)
56     S = 0.5 * np.sum(R * np.roll(Z, -1) - np.roll(R, -1) * Z)
57     Rc = np.sum((R + np.roll(R, -1)) * (R * np.roll(Z, -1) - np.roll(R, -1) * Z)) / (6 * S)
58     return abs(S), 2 * np.pi * Rc * abs(S), Rc
```

Рядки 40–46 — дослівно (4.1). `argmax/argmin` по Z знаходять верхню й нижню точки. Похибка — порядку відстані між точками контуру: для EFIT-контуру це частки сантиметра, для щільно заданої аналітичної кривої вона нехтовно мала. Рядки 53–58 — площа перерізу за формулою Гаусса («шнурка») і об'єм тора за теоремою Паппа–Гульдіна $V = 2\pi R_c S$, де R_c — радіус центроїда перерізу, не R_{geo} .

Порівняння в табл. 4.1 повчальне: за формою перерізу DIII-D і JET дуже схожі ($A \approx 2,4$ – $2,7$, $\kappa \approx 1,7$), а за розміром JET удвічі більший лінійно й у ≈ 8 разів за об'ємом. Відмінність форми — у трикутності: DIII-D асиметричний ($\delta_l \approx 2\delta_u$), бо низ межі проходить через X-точку. JET 1975 симетричний, бо його межу задано аналітично (п. 4.4).

4.2 Форма Міллера

Щоб *задати* межу кількома параметрами (а не прочитати з EFIT), найчастіше беруть параметризацію Міллера [10]:

$$R(t) = R_0 + a \cos(t + \arcsin \delta \sin t), \quad Z(t) = \kappa a \sin t, \quad t \in [0, 2\pi). \quad (4.2)$$

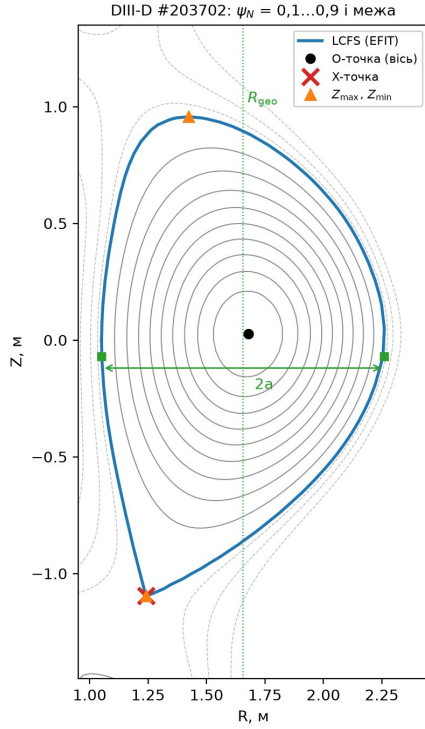


Рис. 4.1. Геометрія DIII-D #203702, кадр 75: поверхні $\psi_N = 0,1 \dots 0,9$ (сірі), LCFS з набору EFIT (синя), відкриті поверхні $\psi_N > 1$ (штрих), магнітна вісь, X-точка і точки, за якими читають параметри (4.1): крайні радіуси (квадрати), найвища й найнижча точки (трикутники). Найнижча точка межі збігається з X-точкою. Вісь зсунута від R_{geo} назовні (зсув Шафранова) і вгору від Z_{geo} , бо X-точка «відтягує» низ межі. Власний розрахунок, код: `geometry/code/geom_tour.py`; дані — Fusion Equilibrium Challenge, CC BY 4.0.

Табл. 4.1. Параметри форми двох прикладів (запуск `geom_tour.py`)

	DIII-D #203702, кадр 75	JET 1975 (межа Міллера)
$R_{\text{geo}}, \text{ м}$	1,656	2,960
$a, \text{ м}$	0,605	1,250
$A = R_{\text{geo}}/a$	2,74	2,37
κ	1,70	1,68
δ_u / δ_l	0,38 / 0,69	0,348 / 0,348
$Z_{\text{geo}}, \text{ м}$	-0,070	0
площа перерізу $S, \text{ м}^2$	1,77	8,12
радіус центроїда $R_c, \text{ м}$	1,591	2,852
об'єм $V, \text{ м}^3$	17,7	145,5
конфігурація	діверторна, нижня X-точка	лімітерна

Перевірка: на $t = 0$ і π маємо $R = R_0 \pm a$, тобто $R_{\text{geo}} = R_0$ і півширина a . На $t = \pi/2$ маємо $Z = \kappa a$ і $R = R_0 + a \cos(\pi/2 + \arcsin \delta) = R_0 - a\delta$. Отже, κ і δ у (4.2) — *точно* ті самі, що в (4.1); для JET скрипт відтворює їх до останнього знака. Повна модель Міллера містить ще радіальні похідні (зсув Шафранова $\partial R_0 / \partial r$, шир витягнутості й трикутності), але для однієї межі досить (4.2).

`viz/src/tokviz/surfaces.py`, рядки 150–166 (D-переріз Міллера)

```

150 def d_shape(R0: float, a: float, kappa: float = 1.8, delta: float = 0.45,
151            n: int = 160) -> np.ndarray:
152     """Miller-parametrised D cross-section, in (R, Z).
153
154      $R(t) = R_0 + a \cos(t + \arcsin(\delta) \sin t)$ 
155      $Z(t) = \kappa a \sin(t)$ 
156
157     This is the standard shaping parametrisation (elongation ``kappa``,
158     triangularity ``delta``) used throughout tokamak physics. It is NOT the
159     constant-tension 'Princeton D' -- that is a different curve, defined by a
160     force-balance condition rather than by shaping parameters.
161     """
162     t = np.linspace(0.0, TWO_PI, int(n), endpoint=False)
163     x = np.arcsin(np.clip(delta, -1.0, 1.0))

```

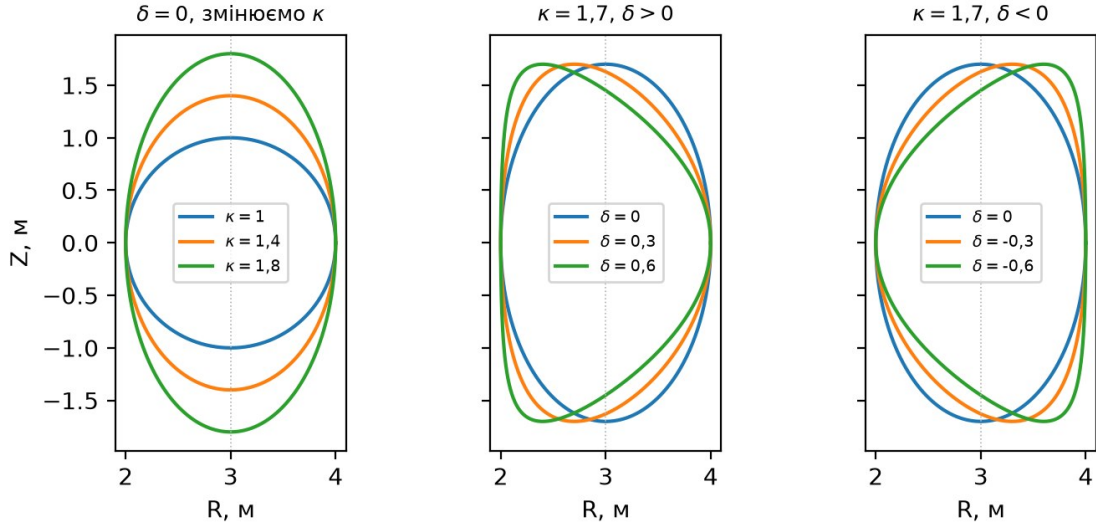



Рис. 4.2. Форма Міллера (4.2) при $R_0 = 3$ м, $a = 1$ м. Ліворуч: витягнутість при $\delta = 0$ (еліпси). У центрі й праворуч: додатна й від’ємна трикутність при $\kappa = 1,7$. Власний розрахунок, функція `tokviz-surfaces.d_shape`.

```

164 R = R0 + a * np.cos(t + x * np.sin(t))
165 Z = kappa * a * np.sin(t)
166 return np.stack([R, Z], axis=-1)

```

Зверніть увагу на застереження в рядках 158–160: D Міллера — не та «D принстонського типу», якої набуває котушка TF. Котушку визначає умова чистого натягу (п. 6.2), а не параметри форми. Схожість букв — збіг: плазмі D-форму дають котушки полоїдального поля, котушці — магнітні сили.

4.3 Навіщо плазмі форма

Більший струм за того самого q . Для еліптичного перерізу у циліндричному наближенні (стандартна оцінка, поза корпусом)

$$q_{\text{cyl}} \approx \frac{2\pi a^2 B_0}{\mu_0 R_0 I_p} \cdot \frac{1 + \kappa^2}{2}. \quad (4.3)$$

При $\kappa = 1,7$ множник $(1 + \kappa^2)/2 = 1,95$: за тієї самої межі стійкості q плазма може нести майже вдвічі більший струм, а утримання в токамаках зростає зі струмом. Проєкт JET оцінював цей виграв у 1,7 раза [1, розд. 5]. Точніша формула для D-перерізу з тороїдальною поправкою — [1, розд. 5, (5.4)]; вона дає для JET $q(a) = 5,99$ (розд. 6).

Ціна: вертикальна нестійкість. Щоб витягнути плазму, потрібне зовнішнє поле з від’ємним індексом спаду; тоді плазма вертикально нестійка і потребує пасивної стінки та активного керування [1, розд. 4].

Трикутність і дивертор. За стандартною теорією (поза корпусом) трикутність підвищує межі стійкості за тиском, а X-точка на кінці «трикутника» дає дивертор. Тому в DIII-D δ_1 більша, ніж δ_n . Від’ємна трикутність — окремий напрям досліджень [1, розд. 4].

Заповнення камери. Кругла плазма в D-подібній камері лишає порожніми «кути». За тієї самої площі поверхні D-переріз вміщує більший об’єм; саме так аргументували форму JET [1, розд. 5].

4.4 Як задали межу JET 1975

Проект JET дає $R_0 = 2,96$ м, $a = 1,25$ м і напіввисоту $b = 2,10$ м (тобто $\kappa = b/a = 1,68$), але не дає трикутності. Натомість у ньому є таблиця 30 точок межі у верхній зовнішній чверті [8, с. 332]. Проект [6] підганяє до неї форму Міллера і бере з підгонки лише δ :

viz/src/tokviz/machine/components/plasma.py, рядки 69–82 (підгонка трикутності до таблиці межі)

```
69 def fit_table_delta(table: np.ndarray, r_in: float) -> dict:
70     t = np.linspace(-0.1, np.pi / 2 + 0.7, 3000)
71
72     def rms(q):
73         a, k, d = q
74         if a <= 0.3 or k <= 0.5 or not (-0.95 < d < 0.95):
75             return 1e3
76         c = miller(r_in + a, a, k, d, t)
77         return float(np.sqrt(np.mean(point_seg_dist(table, c, closed=False) ** 2)))
78
79     best, val = _nelder_mead(rms, [1.25, 1.7, 0.4], [0.1, 0.1, 0.1])
80     best, val = _nelder_mead(rms, best, [0.02, 0.02, 0.02])
81     return {"a_t": float(best[0]), "kappa_t": float(best[1]), "delta": float(best[2]),
82           "R0_t": float(r_in + best[0]), "rms_m": val}
```

Підгонка (a_t, κ_t, δ) методом Нелдера–Міда за середньоквадратичною відстанню від 30 точок таблиці до кривої Міллера (рядки 72–77). Крива на внутрішньому екваторі проходить через край лімітера $R = 1,71$ м: центр $r_{in} + a$ (рядок 76). Рядки 79–80: два проходи з меншим кроком симплекса.

viz/src/tokviz/machine/components/plasma.py, рядки 91–118 (межа плазми JET)

```
91 def plasma_boundary(tr, n_pol: int = 256) -> dict:
92     R0, a, b = tr.v("plasma.R0"), tr.v("plasma.a"), tr.v("plasma.b")
93     tr.v("plasma.elongation_b_over_a"), tr.v("plasma.R_outer_plasma")
94     r_in = tr.v("plasma.R_inner_plasma")
95     tr.v("plasma.boundary_table_Zmax")
96     table = np.asarray(tr.profile("plasma_boundary"), float)[: , :2]
97     wall = np.asarray(tr.profile("vessel_inner"), float)
98     c_min = float(tr.k("plasma_min_wall_clearance"))
99     kappa = b / a
100     fit = fit_table_delta(table, r_in)
101     delta = float(np.round(fit["delta"], 4))
102     t = np.linspace(0, 2 * np.pi, n_pol, endpoint=False)
103     prof = miller(R0, a, kappa, delta, t)
104     ok, dmin, where = clearance(prof, wall)
105     clamped = False
106     if not ok or dmin < c_min: # reduce |delta| step-wise until it fits
107         clamped = True
108         for d in np.linspace(delta, 0.0, 200):
109             prof = miller(R0, a, kappa, d, t)
110             ok, dmin, where = clearance(prof, wall)
111             if ok and dmin >= c_min:
112                 delta = float(d)
113                 break
114     gaps = {"outboard_midplane": float(tr.v("vessel.rs_R_inner_wall_outboard") - (R0 + a)),
115           "inboard_midplane": float((R0 - a) - tr.v("vessel.rs_R_inner_wall_inboard"))}
116     return {"profile": prof, "R0": R0, "a": a, "b": b, "kappa": kappa, "delta": delta,
117           "fit": fit, "clamped": clamped, "min_wall_clearance": dmin,
118           "min_clearance_at": [float(where[0]), float(where[1])], "gaps": gaps}
```

Рядки 99–103: з підгонки береться лише δ (округлена до 10^{-4}), а R_0 , a , κ — з таблиці основних параметрів проекту. Рядки 104–113: перевірка, що плазма не перетинає стінку камери. Якщо зазор менший за допустимий, $|\delta|$ зменшують кроками. Рядки 114–115: зазори на екваторі. Результат (маніфест data/bake_jet1975/manifest.json): $\delta = 0,3476$ при середньоквадратичному відхиленні від таблиці 1,7 см; сама підгонка дає $\kappa_t = 1,745$, тобто таблиця витягнута за b/a . Зазори до стінки на екваторі — 17,9 см ззовні й 4,6 см всередині, найменший зазор по всьому периметру — 3,5 см, обмеження δ не спрацювало. Сама таблиця суперечлива: її верхня точка на 9,5 см вища за b і виходить за стінку, тому таблицю використано лише як *форму*, а не як межу.

Практикум — форма в часі

(1) Для всіх кадрів DIII-D #203702 з `efit_lcfs_r/z` порахуйте `shape_params` і побудуйте $\kappa(t)$, $\delta_u(t)$, $\delta_l(t)$. Коли з'являється X-точка (стрибок δ_l)? (2) Те саме для MAST `mast_shot_28348`: яке аспектне відношення має сферичний токамак? (3) Для межі Міллера JET порівняйте S і V з еліпсом тих самих a , κ (див. задачу 3).

5. Топологія: О-точка, Х-точка, сепаратриса

5.1 Критичні точки і гесіан

Поверхні $\psi = \text{const}$ «ламаються» лише там, де $\nabla\psi = 0$, тобто де $B_p = 0$. Такі точки називають критичними. Їхній тип визначає гесіан $H_\psi = \begin{pmatrix} \psi_{RR} & \psi_{RZ} \\ \psi_{RZ} & \psi_{ZZ} \end{pmatrix}$:

- $\det H_\psi > 0$ — **О-точка** (еліптична): екстремум ψ , навколо — вкладені замкнені контури. Це магнітна вісь або центр магнітного острова.
- $\det H_\psi < 0$ — **Х-точка** (гіперболічна, сідло): крізь неї проходять дві гілки лінії рівня, що перетинаються. Ця лінія рівня — *сепаратриса*.

Докладніше (індекс Пуанкаре–Хопфа, теорія Морса) — [1, розд. 4, п. 4.4]. Скрипт гайду шукає Х-точку як мінімум $|\nabla\psi|^2$ на сплайні і рахує $\det H_\psi$ з других похідних сплайна:

geo/geom_tour.py, рядки 72–85 (Х-точка і гесіан)

```
72 def find_x_point(eq: Equilibrium, r0, z0):
73     """Сідло  $\psi$  біля  $(r0, z0)$ : мінімум  $|\text{grad } \psi|^2$  на сплайні."""
74     f = lambda p: float(eq.dpsi_dR(*p) ** 2 + eq.dpsi_dZ(*p) ** 2)
75     res = minimize(f, [r0, z0], method="Nelder-Mead",
76                   options=dict(xatol=1e-8, fatol=1e-16, maxiter=4000))
77     return float(res.x[0]), float(res.x[1])
78
79
80 def hessian_det(eq: Equilibrium, r, z):
81     s = eq._spl
82     prr = float(s.ev(r, z, dx=2, dy=0))
83     pzz = float(s.ev(r, z, dx=0, dy=2))
84     prz = float(s.ev(r, z, dx=1, dy=1))
85     return prr * pzz - prz ** 2
```

Рядок 74: цільова функція $|\nabla\psi|^2$ має мінімум нуль у *будь-якій* критичній точці, тож стартова точка вирішує, яку з них знайде метод. У рядку 101 файлу старт — найнижча точка LCFS. Рядки 81–85: $\det H_\psi$ з аналітичних других похідних бікубічного сплайна. На сітці з кроком 2,7–5 см це оцінка, а не точне значення.

5.2 Магнітна вісь зблизька

Біля О-точки $\psi - \psi_{ax} \approx \frac{1}{2} \mathbf{x}^T H_\psi \mathbf{x}$, де $\mathbf{x} = (R - R_{ax}, Z - Z_{ax})$: поверхні — подібні еліпси. Звідси два «локальні» параметри геометрії.

- *Витягнутість на осі* $\kappa_0 = \sqrt{\psi_{RR}/\psi_{ZZ}}$ (при $\psi_{RZ} \approx 0$).
- *q на осі*. Еліпс рівня $\delta\psi$ має площу $2\pi \delta\psi / \sqrt{\det H_\psi}$. Тороїдальний потік через нього на радіан $\psi_t = (F/R_{ax}) \cdot \text{площа}/2\pi$, тому

$$q_0 = \frac{d\psi_t}{d\psi} = \frac{F}{R_{ax} \sqrt{\det H_\psi}}. \quad (5.1)$$

Цю формулу використовує `SolovevEquilibrium.q_axis` для JET (розд. 6).

Для DIII-D, кадр 75: $\psi_{RR} = 3,47$, $\psi_{ZZ} = 2,29$, $\psi_{RZ} = -0,04$ Вб/(рад·м²), $\det H_\psi = 7,95 > 0$. Отже, $\kappa_0 = 1,23$ проти $\kappa = 1,70$ на межі: *витягнутість зростає від осі до краю*. Це типово, бо форму задають зовнішні котушки, і їхній вплив згасає вглиб плазми. За (5.1) $q_0 = 0,68$, тоді як контурний інтеграл на $\psi_N = 0,02$ дає 0,711. Різниця $\approx 4\%$ — це межа точності других похідних сплайна на сітці 65×65 біля осі, а не фізика.

5.3 Х-точка і сепаратриса

Для кадру 75 скрипт знаходить Х-точку в $(R, Z) = (1,2407 \text{ м}, -1,0962 \text{ м})$. Там $B_p = 1,6 \cdot 10^{-9}$ Тл (чисельний нуль) і $\psi_N = 1,00002$: Х-точка лежить на LCFS і збігається з її найнижчою точкою

(рис. 4.1). Це однопольова диверторна конфігурація (lower single null): межу плазми задає не стінка, а сепаратриса.

Гесіан у X-точці: $\psi_{RR} = 0,463$, $\psi_{ZZ} = -0,497$, $\psi_{RZ} = -0,546$; власні значення $-0,743$ і $+0,710$, $\det H_\psi = -0,53 < 0$ — сідло. Звідси маленька, але гарна перевірка фізики.

Виведення — чому гілки сепаратриси перетинаються під прямим кутом

У X-точці $\psi_R = 0$, тому оператор Г'рета-Шафранова $\Delta^*\psi = \psi_{RR} - \psi_R/R + \psi_{ZZ}$ зводиться до сліду гесіана, а рівняння рівноваги [1, розд. 3] дає $\psi_{RR} + \psi_{ZZ} = -\mu_0 R_X J_\varphi$, де R_X — радіус X-точки. На сепаратрисі струму майже немає, тож слід малий, і власні значення протилежні: $\lambda_1 \approx -\lambda_2$. Нульова лінія квадратичної форми $\lambda_1 u^2 + \lambda_2 v^2 = 0$ — це пара прямих $v = \pm \sqrt{-\lambda_1/\lambda_2} u$, і при $\lambda_1 = -\lambda_2$ вони перетинаються під кутом 90° . Для кадру 75 слід дорівнює $-0,034$ (5 % від $|\lambda|$), кут між гілками $\approx 91^\circ$, а $|J_\varphi| \approx 0,034/(\mu_0 \cdot 1,24 \text{ м}) \approx 2 \cdot 10^4 \text{ А/м}^2$. Для порівняння, середня густина струму $I_p/S = 1,36 \text{ МА/1,77 м}^2 \approx 7,7 \cdot 10^5 \text{ А/м}^2$ ($|I_p| = 1,359 \text{ МА}$ — за законом Ампера, циркуляцією B_p уздовж LCFS, функція `ip_ampere` у `geom_tour.py`, рядки 64–69), тобто в X-точці лише $\sim 3\%$ від середньої. Точність других похідних тут та сама, що й на осі, тому число — порядок величини.

Навколо сепаратриси простір поділено на три області:

- *всередині LCFS* — замкнені вкладені поверхні, «ядро» плазми;
- *SOL* (scrape-off layer) — поверхні $\psi_N > 1$ над X-точкою, відкриті: силова лінія йде вздовж плазми й упирається в пластини дивертора;
- *приватна область* (private flux region) — під X-точкою, між двома «ногами» сепаратриси. Тут також $\psi_N < 1$, хоча плазми ядра там немає. Тому маска « $\psi_N < 1$ » без перевірки «всередині LCFS» захоплює цю область; на рис. 3.1 її довелося вирізати окремо.

Чому $q \rightarrow \infty$ на сепаратрисі. У (3.1) підінтегральна функція $1/(R|\nabla\psi|)$ біля X-точки поводить як $1/(\text{відстань до X})$, і для поверхні, що проходить на відстані d від X-точки, інтеграл росте як $\ln(1/d)$. Лінія, яка проходить поблизу X-точки, «застрягає» там на багато тороїдальних обертів. Звідси крутий ріст q біля межі: $q(0,95) = 3,70$, $q(0,98) = 4,50$ (розд. 3).

5.4 Як знаходять LCFS

EFIT видає LCFS разом з рівновагою, але оцінювач конкурсу Fusion Equilibrium Challenge ви-тягає її сам із будь-якої карти ψ — і справжньої, і передбаченої. Означення: LCFS — найнижчий рівень $\text{AXIS_SIGN} \cdot \psi$, який ще дає *один замкнений контур навколо осі всередині допустимої області машини*. Нижче цього рівня контур або виходить на стінку (лімітер), або розпадається в X-точці (дивертор). Рівень шукають бісекцією:

`fec/starter/fusion_scoring/lcfs.py`, рядки 58–89 (бісекція рівня LCFS)

```
58 def _extract_one_sign(phi: np.ndarray, R: np.ndarray, Z: np.ndarray,
59                       mask_coarse: np.ndarray | None, mask_f: np.ndarray | None,
60                       n_iter: int, n_points: int):
61     if not np.isfinite(phi).all():
62         phi = np.nan_to_num(phi, nan=np.nanmin(phi) - 1.0, posinf=np.nanmax(phi),
63                             neginf=np.nanmin(phi))
64     # Magnetic axis = strongest interior LOCAL maximum of phi (the 0-point), not a global
65     # argmax: on ~1.4% of MAST frames the coil-driven flux at the envelope edge exceeds the
66     # flux at the plasma axis, and a global argmax walks off to the mask rim, after which the
67     # bisection finds no closed surface and the frame is lost. See o_point.py.
68     mask = mask_coarse if (mask_coarse is not None and mask_coarse.any()) else None
69     _, _, iz, ir, _ = find_o_point(phi, R, Z, mask)
70     axis_val = float(phi[iz, ir])
71     border = np.concatenate([phi[0, :], phi[-1, :], phi[:, 0], phi[:, -1]])
72     lo = float(np.min(border)) # low bracket: level that opens onto the wall (invalid)
73     hi = axis_val - 1e-9      # high bracket: tight closed loop around the axis (valid)
74     if not (hi > lo):
75         return None
76     best = _best_contour_at(phi, hi, iz, ir, mask_f)
77     if best is None:
78         return None # even a tight loop is not closed/enclosing -> give up
```

```

79 # Binary-search the lowest still-valid level (the last closed surface).
80 for _ in range(n_iter):
81     mid = 0.5 * (lo + hi)
82     c = _best_contour_at(phi, mid, iz, ir, mask_f)
83     if c is not None:
84         hi = mid
85         best = c
86     else:
87         lo = mid
88 phys = grid_to_physical(best, R, Z)
89 return resample_closed(phys, n_points)

```

Рядки 68–70: вісь — найсильніший *локальний* максимум, а не глобальний (коментар у рядках 64–67: на 1,4 % кадрів MAST поле котушок на краю області сильніше, ніж на осі). Рядки 71–73: нижня межа бісекції — мінімум ϕ на краю сітки (там контур точно відкритий), верхня — трохи нижче осі (там точно малий замкнений контур). Рядки 80–87: 22 кроки бісекції звужують інтервал у $2^{22} \approx 4 \cdot 10^6$ разів. `_best_contour_at` (рядки 44–55) приймає контур, лише якщо він замкнений, охоплює вісь і лежить у масці камери. Цей алгоритм однаково працює для лімітерної і диверторної плазми, бо перевіряє саме топологію (замкненість і охоплення), а не тип межі.

5.5 Скелет критичних точок як критерій якості

Справжня рівновага всередині LCFS має рівно одну О-точку і жодної Х-точки. Шум у передбаченій ψ народжує зайві пари О–Х, тобто острови, яких немає. Проєкт використав це як незалежний критерій якості передбачення; повний розбір із числами (E9–E16, R5, R7, R9) є в методичці [1, розд. 4, п. 4.4] і путівнику [2]. Тут — лише код, що рахує індекс:

`ourtry/04-novelty/topology_probe.py`, рядки 63–90 (критичні точки через число обертів $\nabla\psi$)

```

63 from scipy.ndimage import label
64 dR, dZ = R[1] - R[0], Z[1] - Z[0]
65 gz, gr = np.gradient(psi, dZ, dR)
66 ang = np.arctan2(gz, gr)
67 nz, nr = psi.shape
68
69 raw = np.zeros_like(psi)
70 for iz in range(1, nz - 1):
71     for ir in range(1, nr - 1):
72         if not inside[iz, ir]:
73             continue
74         a = [ang[iz + dz, ir + dr] for dz, dr in LOOP]
75         a.append(a[0])
76         d = np.diff(a)
77         d = (d + np.pi) % (2 * np.pi) - np.pi
78         raw[iz, ir] = np.round(d.sum() / (2 * np.pi))
79
80 lab, n = label(raw != 0, structure=np.ones((3, 3)))
81 out = []
82 for c in range(1, n + 1):
83     zs, rs = np.where(lab == c)
84     iz0, iz1 = max(zs.min() - 1, 0), min(zs.max() + 1, nz - 1)
85     ir0, ir1 = max(rs.min() - 1, 0), min(rs.max() + 1, nr - 1)
86     w = _winding(ang, iz0, iz1, ir0, ir1)
87     if w != 0:
88         out.append({"iz": float(zs.mean()), "ir": float(rs.mean()), "index": w,
89                     "npix": int(len(zs))})
90 return out

```

Рядки 65–66: напрямок $\nabla\psi$ у кожному вузлі. Рядки 69–78: для кожного вузла — число обертів цього напрямку по 8 сусідах (+1 для О-точки, −1 для Х-точки, 0 для звичайної точки); стрибки кута зводяться в $(-\pi, \pi]$ (рядок 77). Рядки 80–89: одна справжня критична точка «засвічує» блок $\approx 2 \times 2$ вузли, тому сусідні вузли об'єднують у компоненту і рахують число обертів по периметру її обмежувального прямокутника. Так кожену точку враховано один раз.

Встановлено в проєкті [E9–E11]

На кадрах DIII-D #203702 без шуму скелет — 1 О-точка, 0 Х-точок, сума індексів $+1$. Скелет стійкий до шуму 3% від $\text{std}(\psi)$. При 10% з'являється ≈ 20 зайвих точок, але сума індексів лишається $\approx +1$, бо точки народжуються парами О–Х. Тому інформативна *кількість* точок, а не їхня сума [3], [1, розд. 4].

Практикум — топологія

(1) Запустіть `find_x_point` зі старту на вісі. Що знайде метод і чому? (2) Знайдіть другу (верхню) критичну точку DIII-D поза LCFS, стартувавши над верхньою точкою межі. Яке в неї ψ_N і $\det H_\psi$? Наскільки розряд далекий від double null? (3) У `topology_probe.py` вимкніть ерозію (`erode=0`) і поясніть, чому сума індексів стає $1 - k_X$.

Коди перерізу:

- мідь (обмотки TF, PF)
- залізо (ядро)
- Inconel (камера, сільфони, порти)
- сталь (корпуси, опори, циліндр)
- алюмінієвий сплав (кільце, оболонка)
- Мо / графіт / Ni (лімітери)
- плазма: ψ_N , тепле ядро → холодний край

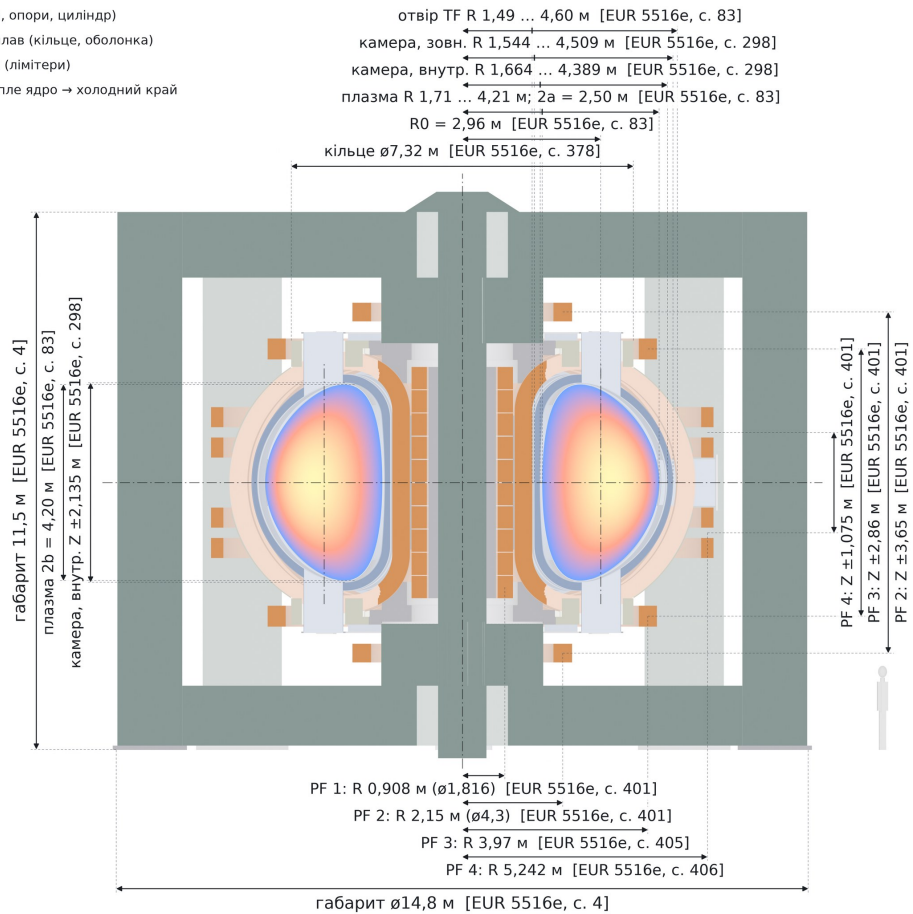


Рис. 6.1. Вертикальний переріз JET 1975 з розмірами: від осі назовні — центральна колона ярма, стек PF-1, внутрішня нога TF, камера, плазма (ψ_N), зовнішня нога TF, PF-2-4, вертикальні плечі. Власна 3D-реконструкція [6] (рисунок з методички [1, розд. 5]); побудовано за даними [8].

6. Геометрія JET: від плазми до котушок

Попередні розділи йшли від ψ до форми. Тут — навпаки: від вимог до форми машини. Приклад — проєкт JET 1975 р. [8], за яким проєкт має 3D-реконструкцію [6]. Повна «анатомія» з механікою, камерою, трансформатором і портами — у методичці [1, розд. 5]. Тут лише те, що стосується геометрії, і код, що її рахує.

6.1 Аспектне відношення як компроміс

Струм котушок TF, потрібний для заданих I_p і q , зростає як $(R_0/a)^2$ [1, розд. 5, (5.1)], тому магніт дешевшає зі зменшенням A . Знизу A обмежує центральний стовп: крізь отвір тора треба провести внутрішні ноги котушок TF і осердя трансформатора, яке наводить струм. Проєкт знайшов мінімум $A \approx 2,2$ і обрав $R_0/a = 2,37$ [1, розд. 5]. Табл. 4.1 показує той самий порядок у DIII-D ($A = 2,74$). Ціна малого A — більше захоплених частинок: для зовнішньої поверхні JET $\sqrt{2\epsilon/(1+\epsilon)} = 0,77$ при $\epsilon = a/R_0 = 0,42$ [1, розд. 5].

6.2 Котушка TF: D чистого натягу

Котушка TF розтягується магнітним тиском назовні. Кругла котушка при цьому згинається, а згин руйнує ізоляцію між витками. D-подібна котушка сприймає навантаження *лише натягом*,

без згину. Умова цього — радіус кривини контуру, пропорційний R [1, розд. 5, (5.3)]:

$$\rho(R) = k_D R, \quad k_D = \frac{1}{2} \ln \frac{R_2}{R_1}, \quad (6.1)$$

де R_1 — верх прямої внутрішньої ноги, R_2 — зовнішній екватор. Для осьової лінії обмотки нашої моделі $R_1 = 1,304$ м, $R_2 = 4,790$ м [1, розд. 5], звідки $k_D = 0,651$: радіус кривини 0,85 м біля внутрішньої ноги і 3,12 м на зовнішньому екваторі. Контур круто загинається всередині і полого — зовні; разом із прямою ногою це й дає літеру D. Як уже сказано в п. 4.2, це *інша* крива, ніж D Міллера для плазми (див. також VR6: проєкт свідомо не наводить замкненої формули кривої Princeton D [4]). У коді котушку не будують з (6.1): контур отвору оцифровано з креслення проєкту (machines/jet1975/profiles/tf_coil_inner.csv) і зсунуто на половину товщини ноги.

6.3 Гофрування торіодального поля

32 окремі котушки порушують осесиметрію: $|B|$ стає періодичним по φ з періодом $2\pi/32$. Глибину гофрування рахують так: $\delta_r = (B_{\max} - B_{\min}) / (B_{\max} + B_{\min})$, де $B_{\max}$ — у площині котушки, $B_{\min}$ — посередині між котушками. *Пастка*: проєкт JET означає $\varepsilon_r = 2\delta_r$ [1, розд. 5]. Код рахує обидва варіанти за законом Біо–Савара від 32 ниток уздовж осьової лінії обмотки:

viz/src/tokviz/machine/ripple.py, рядки 122–137 (гофрування за Біо–Саваром)

```
122 def ripple(loops, R, Z=0.0, n_coils: int = 32, phi0: float = 0.0):
123     """Return ``(delta, eps_src, Bmax, Bmin)`` at (R, Z) arrays.
124
125     Uses |B| on the coil plane and midway between coils. (On the midplane
126     B_R = B_Z = 0 by symmetry, so |B| = |B_phi| there.)
127     """
128     R = np.atleast_1d(np.asarray(R, float))
129     Z = np.broadcast_to(np.asarray(Z, float), R.shape)
130     out = []
131     for ph in (phi0, phi0 + np.pi / n_coils):
132         BR, BP, BZ = field_rz_phi(loops, R, Z, np.full(R.shape, ph))
133         out.append(np.sqrt(BR ** 2 + BP ** 2 + BZ ** 2))
134     B_on, B_off = out
135     Bmax, Bmin = np.maximum(B_on, B_off), np.minimum(B_on, B_off)
136     d = (Bmax - Bmin) / (Bmax + Bmin)
137     return d, 2 * d, Bmax, Bmin
```

Рядки 131–133: $|B|$ у площині котушки (φ_0) і посередині між котушками ($\varphi_0 + \pi/32$) у тих самих точках (R, Z) . Рядки 135–137: обидва означення. Відношення не залежить від струму, бо поле лінійне за ним.

Модель проєкту — гофрування JET

Запуск `geom_tour.py` (41 МА, одна нитка на котушку): на краю плазми $R = 4,21$ м $\delta_r = 1,83\%$, тобто $\varepsilon_r = 3,67\%$ проти проєктних $3,6\%$; на осі R_0 $\delta_r = 3 \cdot 10^{-7}$; $B(R_0) = 2,770$ Тл, як у проєкті (2,77). Похибку дає переважно оцифрування контуру котушки: зсув осьової лінії на ± 2 см змінює $\varepsilon_r(4,21 \text{ м})$ на $+14/-12\%$ [1, розд. 5].

Чому гофрування так круто зростає назовні? Грубо (поза корпусом): поле дискретних ниток з кроком $2\pi R_2/N$ на радіусі R_2 спадає всередину як $(R/R_2)^N$. Для $N = 32$ оцінка $(4,21/4,79)^{32} = 1,6\%$ близька до $1,83\%$ за Біо–Саваром. Плазма JET займає отвір майже до зовнішньої ноги, і за це доводиться платити гофруванням.

6.4 Рівновага в межі JET

Щоб побачити магнітні поверхні всередині D-межі (п. 4.4), потрібна рівновага. Найпростіша точна — рівновага Соловйова: при сталих $\mu_0 p'$ і FF' рівняння Греда–Шафранова лінійне. Його розв'язок — частинний поліном плюс сім однорідних розв'язків [9]. Сім коефіцієнтів задають

так, щоб межа $\psi = 0$ проходила через три точки D (зовнішній і внутрішній екватор, верх) з правильним нахилом і кривинами Міллера:

viz/src/tokviz/machine/equilibrium_analytic.py, рядки 283–303 (сім умов Cerfon–Freidberg)

```

283 eps = a / R0
284 al = np.arcsin(delta)
285 up = _P((1 / 8, 4, 0, 0)) + A * _P((0.5, 2, 0, 1), (-1 / 8, 4, 0, 0))
286 if method == "cf7":
287     basis = CF_HOMOGENEOUS
288     N1 = -(1 + al) ** 2 / (eps * kappa ** 2)
289     N2 = (1 - al) ** 2 / (eps * kappa ** 2)
290     N3 = -kappa / (eps * np.cos(al) ** 2)
291     xo, xi, xt, yt = 1 + eps, 1 - eps, 1 - delta * eps, kappa * eps
292
293     def rows(f):
294         fx, fy = f.dx(), f.dy()
295         fxx, fyy = fx.dx(), fy.dy()
296         return np.array([
297             f(xo, 0.0), f(xi, 0.0), f(xt, yt), fx(xt, yt),
298             fyy(xo, 0.0) + N1 * fx(xo, 0.0),
299             fyy(xi, 0.0) + N2 * fx(xi, 0.0),
300             fxx(xt, yt) + N3 * fy(xt, yt)], float)
301
302     M = np.stack([rows(h) for h in basis], axis=1)
303     c = np.linalg.solve(M, -rows(up))

```

Рядки 283–285: $\epsilon = a/R_0$, $\arcsin \delta$ і частинний розв’язок u_p (A — параметр поділу струму між тиском і FF'). Рядки 288–290: кривини межі Міллера на зовнішньому екваторі, внутрішньому екваторі й у верхній точці. Рядок 291: ці три точки в безрозмірних координатах $x = R/R_0$, $y = Z/R_0$. Рядки 296–300: сім умов — $u = 0$ у трьох точках, $\partial u / \partial x = 0$ у верхній точці (там дотична горизонтальна) і три умови кривини. Рядок 303: лінійна система 7×7 .

Лишаються два вільні параметри. Параметр A підбирають бісекцією так, щоб $\beta_p = 0,9$ (як у виносках проекту), а масштаб ψ_0 — так, щоб $I_p = 3,8$ МА:

viz/src/tokviz/machine/equilibrium_analytic.py, рядки 490–517 (збирання рівноваги JET)

```

490 def build_equilibrium(R0=JET_R0, a=JET_A, b=JET_B, delta=JET_DELTA, B0=JET_B0,
491                     Ip=JET_IP, beta_p=JET_BETAP, A=None, method="cf7",
492                     grid=(193, 257), margin=(0.24, 0.30)):
493     """Solov'ev JET equilibrium: shape from G2, B0 and I_p from Table I.3-1.
494
495     ``A`` overrides the beta_p solve. ``grid = (nR, nZ)``; the grid extends
496     ``margin = (dR, dZ)`` metres beyond the plasma so the psi_n = 1 contour
497     and field lines on it stay well inside the spline domain.
498
499     Returns ``(eq, sol, info)``.
500     """
501     kappa = b / a
502     if A is None:
503         A = solve_A_for_beta_p(R0, a, kappa, delta, beta_p, method=method)
504     sol = solve_shape(R0, a, kappa, delta, A, method=method)
505     sol.B0 = B0
506     scale_to_current(sol, Ip)
507     R = np.linspace(R0 - a - margin[0], R0 + a + margin[0], grid[0])
508     Z = np.linspace(-b - margin[1], b + margin[1], grid[1])
509     eq = SolovevEquilibrium(sol, R, Z)
510     gq = global_quantities(sol)
511     res_max, res_rms = sol.boundary_residual()
512     info = {"A": float(A), "psi0_Wb_per_rad": sol.psi0, "method": method,
513            "n_basis": len(sol.basis), "kappa": kappa, "delta": delta,
514            "boundary_residual_max_m": res_max, "boundary_residual_rms_m": res_rms,
515            "C1_minus_mu0_pprime": sol.C1, "C2_minus_FFprime": sol.C2,
516            "F0_Tm": sol.F0, **gq}
517     return eq, sol, info

```

Рядки 501–506: $\kappa = b/a$, бісекція по A для заданої β_p , розв’язок форми і масштаб ψ_0 під I_p . Рядки 507–509: ψ розраховують на сітці 193×257 із запасом за межею плазми і загортають у

Табл. 6.1. Рівновага Соловйова в межі JET 1975 (запуск `geom_tour.py`; ті самі числа — у `data/bake_jet1975/physics.json` [6])

A (поділ струму)	0,0948	I_p	3,80 МА
ψ_0	15,64 Вб/рад	β_p / β_t	0,900 / 2,32 %
нев'язка межі	0,08 мм	$l_i(1) / l_i(3)$	0,498 / 0,425
вісь R_{ax}	3,183 м	зсув Шафранова $R_{ax} - R_0$	0,223 м
q_0 (з гесіана)	2,80	$q_{95} / q(0,99)$	5,51 / 5,75
об'єм (квадратура)	145,46 м ³	об'єм (Папп, табл. 4.1)	145,46 м ³

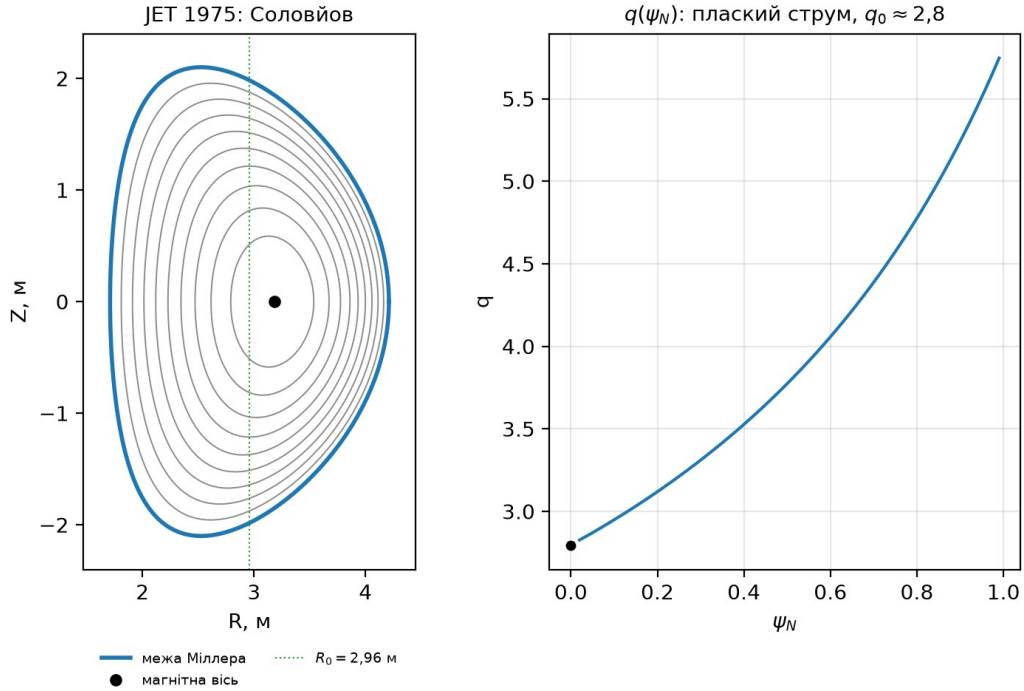


Рис. 6.2. Рівновага Соловйова в межі JET 1975. Ліворуч: поверхні $\psi_N = 0,1 \dots 0,9$, межа Міллера ($\kappa = 1,68$, $\delta = 0,348$) і магнітна вісь, зсунута на 0,22 м назовні від R_0 . Праворуч: $q(\psi_N)$ з контурного інтеграла (3.1) і q_0 з гесіана (5.1) (точка). Власний розрахунок, код: `geometry/code/geom_tour.py`.

той самий клас `Equilibrium`, що й кадри EFIT. Тому весь код розд. 2–5 (поверхні, q , θ^*) працює для JET без змін.

Об'єм, обчислений квадратурою по сітці, збігається з об'ємом за Паппом (п. 4.1) з відносною різницею 10^{-5} — незалежна перевірка обох кодів. Контурний q і q з трасування силових ліній збігаються до 0,015 % на $\psi_N = 0,1 \dots 0,95$ (`physics.json`, ключ `q_traced_vs_contour`). Помітний і зсув Шафранова: вісь на 0,22 м зовні від R_0 (рис. 6.2), а поверхні згущуються до зовнішнього боку.

Чесне обмеження. Струм Соловйова пласкої форми: $J_\varphi = (C_1 R + C_2/R)/\mu_0$ від ψ не залежить. Тому $q_0 = 2,80$ і $q(a)/q_0 \approx 2,1$, тоді як проєкт розраховував на пікований струм з $q_0 \approx 1$. Поверхонь $q = 1, 3/2, 2$ у цій моделі немає. Для топології (вкладені поверхні, замикання раціональних ліній $q = 3, 7/2, 4, 5$ з точністю $< 0,5$ мм) модель придатна, для МГД-стійкості — ні [1, розд. 5].

Практикум — JET

- (1) Побудуйте рівновагу з `method="lsq"` замість `"cf7"`: як зміняться невязка межі і q_0 ?
- (2) Змініть δ на 0 і на 0,5 (`build_equilibrium(delta=...)`): як залежать q_{95} і зсув Шафранова від трикутності? (3) Порахуйте δ_r на $R = 4,0 \dots 4,4$ м з кроком 5 см і порівняйте з

оцінкою $(R/R_2)^{32}$. Де оцінка найгірша і чому?

7. Практикум і задачі

7.1 Контрольні запитання

1. Чому вся геометрія осесиметричної рівноваги задається однією функцією $\psi(R, Z)$? Що саме порушує осесиметрію в реальній машині?
2. Чому в таблицях і графіках поверхні позначають через ψ_N , а не через ψ ? Що станеться з ψ_N , якщо помножити ψ на -1 ?
3. Чому q_{95} , а не q на межі? Що відбувається з q на сепаратрисі?
4. Чим кут θ^* кращий за геометричний θ для опису островів? Чому лінії сталого θ^* густіші на внутрішньому боці тора?
5. Чому θ^* не залежить від F , а q залежить лінійно?
6. Чим відрізняються D Міллера (плазма) і D чистого натягу (котушка TF)? Що визначає кожну з них?
7. Чому гілки сепаратриси перетинаються майже під прямим кутом? Коли кут відрізнявся б від 90° ?
8. Чому маска « $\psi_N < 1$ » не збігається з областю «всередині LCFS» у диверторній плазмі?

7.2 Задачі

1. *Міллер.* Доведіть, що в параметризації (4.2) верхня точка має $R = R_0 - a\delta$ і $Z = \kappa a$, тобто κ і δ збігаються з означеннями (4.1). Обчисліть $R_{Z_{\max}}$ для JET ($R_0 = 2,96$ м, $a = 1,25$ м, $\delta = 0,3476$).
2. *Виграш від витягнутості.* За (4.3) оцініть q_{cyl} для DIII-D #203702 ($a = 0,605$ м, $\kappa = 1,70$, $R_{\text{geo}} = 1,656$ м, $B_\varphi(R_{\text{geo}}) = 1,953$ Тл, $|I_p| = 1,359$ МА) з урахуванням і без урахування κ . Порівняйте з $q_{95} = 3,70$. Що може пояснити різницю?
3. *Площа й об'єм.* Для JET порівняйте площу й об'єм межі Міллера (табл. 4.1) з еліпсом тих самих a , κ з центром на R_0 . Чому трикутність зменшує об'єм помітніше, ніж площу?
4. *Вісь.* За гесіаном на осі DIII-D ($\psi_{RR} = 3,473$, $\psi_{ZZ} = 2,288$, $\psi_{RZ} = -0,036$ Вб/(рад·м²)), $F = 3,2334$ Т м і $R_{\text{ax}} = 1,6782$ м знайдіть κ_0 і q_0 за (5.1). Виведіть (5.1).
5. *X-точка.* Власні значення гесіана в X-точці DIII-D — $-0,743$ і $+0,710$ Вб/(рад·м²), $R_X = 1,241$ м. Знайдіть кут між гілками сепаратриси і густину струму $|J_\varphi|$ в X-точці. Порівняйте з середньою $|I_p|/S$.
6. *D-котушка.* Для $R_1 = 1,304$ м, $R_2 = 4,790$ м знайдіть k_D і радіуси кривини на кінцях дуги. Наскільки менший радіус біля внутрішньої ноги, ніж на зовнішньому екваторі?
7. *Гофрування.* Оцініть δ_T на $R = 4,21$ м і на $R_0 = 2,96$ м як $(R/R_2)^N$ при $N = 32$. Порівняйте з Біо-Саваром ($1,83\%$ і $3 \cdot 10^{-7}$). Чому на осі проста оцінка непридатна?

Практикум — міні-проект

Напишіть функцію, яка для будь-якого кадру DIII-D чи MAST за 1 с повертає «паспорт геометрії»: R_{ax} , Z_{ax} , R_{geo} , a , A , κ , δ_u , δ_l , S , V , $|I_p|$, координати й ψ_N X-точок, q_0 (з гесіана), q_{95} . Усі цеглинка вже є в `geom_tour.py` і `tokviz.equilibrium`. Перевірте на кадрі 75: паспорт має відтворити числа розд. 2–5.

А. Відповіді і шпаргалка

А.1 Відповіді до задач

1. На $t = \pi/2$: $\sin t = 1$, $Z = \kappa a$ — максимум Z , отже $\kappa = Z_{\max}/a$; $R = R_0 + a \cos(\pi/2 + \arcsin \delta) = R_0 - a \sin(\arcsin \delta) = R_0 - a\delta$, отже $(R_{\text{geo}} - R_{Z_{\max}})/a = \delta$. JET: $R_{Z_{\max}} = 2,96 - 1,25 \cdot 0,3476 = 2,5255$ м (збігається з `geom_tour.json`).
2. З κ : $q_{\text{cyl}} = 3,08$; без κ : 1,59. Реальне $q_{95} = 3,70$ ще на 20 % більше: формула (4.3) не враховує тороїдальності ($\epsilon = 0,37$), трикутності й того, що q_{95} береться поблизу X-точки, де q круто зростає (п. 5.3).
3. Еліпс: $S = \pi a^2 \kappa = 8,25 \text{ м}^2$, $V = 2\pi R_0 S = 153,4 \text{ м}^3$. Міллер: $S = 8,12 \text{ м}^2$ (−1,6 %), $V = 145,5 \text{ м}^3$ (−5,2 %). Трикутність переносить верх і низ перерізу всередину: центроїд зсувається з $R_0 = 2,96$ м на $R_c = 2,852$ м, а $V = 2\pi R_c S$ чутливий до R_c .
4. $\kappa_0 = \sqrt{3,473/2,288} = 1,23$. $\det \mathbf{H}_\psi = 3,473 \cdot 2,288 - 0,036^2 = 7,95$; $q_0 = 3,2334/(1,6782 \cdot \sqrt{7,95}) = 0,68$. Виведення: еліпс $\frac{1}{2} \mathbf{x}^T \mathbf{H} \mathbf{x} = \delta \psi$ має площу $2\pi \delta \psi / \sqrt{\det \mathbf{H}}$; тороїдальний потік на радіан $\psi_t = B_\varphi \cdot \text{площа} / 2\pi = F \delta \psi / (R_{\text{ax}} \sqrt{\det \mathbf{H}})$; $q = d\psi_t / d\psi$.
5. Нульова лінія $\lambda_1 u^2 + \lambda_2 v^2 = 0$: $v/u = \pm \sqrt{0,743/0,710} = \pm 1,023$, кут кожної гілки до осі u — $45,7^\circ$, між гілками — $91,3^\circ$. $|J_\varphi| = |\lambda_1 + \lambda_2| / (\mu_0 R_X) = 0,033 / (1,257 \cdot 10^{-6} \cdot 1,241) \approx 2,1 \cdot 10^4 \text{ А/м}^2$; $|I_p|/S = 1,359 \cdot 10^6 / 1,769 = 7,7 \cdot 10^5 \text{ А/м}^2$, відношення $\approx 3\%$.
6. $k_D = \frac{1}{2} \ln(4,790/1,304) = 0,651$; $\rho(R_1) = 0,85$ м, $\rho(R_2) = 3,12$ м — у $R_2/R_1 = 3,67$ раз менший.
7. $(4,21/4,79)^{32} = 1,6\%$ проти $1,83\%$ — непогано. На осі $(2,96/4,79)^{32} = 2 \cdot 10^{-7}$ — порядок збігається випадково: на осі вже важать внески внутрішніх ніг ($\propto (R_1/R)^{32}$) і скінчення довжина контуру, яких оцінка не містить.

Практикуми. Розд. 5, п. (1): зі старту біля осі метод знаходить саму вісь (1,67817; 0,02623) м — у ній теж $|\nabla \psi| = 0$. П. (2): верхня критична точка лежить у (1,170 м, 1,149 м), $\psi_N = 1,058$, $\det \mathbf{H}_\psi = -0,32$ — сідло поза LCFS. Розряд однонульовий, але друга X-точка лише на $\Delta \psi_N \approx 0,06$ «зовні». Інші практикуми відкриті; числа — з вашого запуску.

А.2 Шпаргалка

Шпаргалка — формули гайду

Поле	$\mathbf{B} = \nabla \psi \times \nabla \varphi + F \nabla \varphi$; $B_R = -\psi_Z/R$, $B_Z = \psi_R/R$, $B_\varphi = F/R$, $B_p = \nabla \psi /R$
Нормований потік	$\psi_N = (\psi - \psi_{\text{ax}}) / (\psi_b - \psi_{\text{ax}})$
Запас стійкості	$q = \frac{F}{2\pi} \oint \frac{d\ell}{R \nabla \psi }$, $q_0 = \frac{F}{R_{\text{ax}} \sqrt{\det \mathbf{H}_\psi}}$
Кут прямих ліній	$\theta^* = 2\pi \int_0^\ell \frac{d\ell'}{R \nabla \psi } \bigg/ \oint \frac{d\ell'}{R \nabla \psi }$, $d\theta^*/d\varphi = 1/q$
Форма	$R_{\text{geo}} = \frac{R_{\text{max}} + R_{\text{min}}}{2}$, $a = \frac{R_{\text{max}} - R_{\text{min}}}{2}$, $\kappa = \frac{Z_{\text{max}} - Z_{\text{min}}}{2a}$, $\delta_{u,l} = \frac{R_{\text{geo}} - R_{Z_{\text{max,min}}}}{a}$
Міллер	$R = R_0 + a \cos(t + \arcsin \delta \sin t)$, $Z = \kappa a \sin t$
Площа, об'єм	$S = \frac{1}{2} \sum (R_i Z_{i+1} - R_{i+1} Z_i)$, $V = 2\pi R_c S$
Струм плазми	$I_p = \mu_0^{-1} \oint B_p \cdot d\ell$ по LCFS
Критичні точки	$\nabla \psi = 0$; $\det \mathbf{H}_\psi > 0$ — O, < 0 — X; у X-точці $\psi_{RR} + \psi_{ZZ} = -\mu_0 R J_\varphi$
D-котушка	$\rho = k_D R$, $k_D = \frac{1}{2} \ln(R_2/R_1)$
Гофрування	$\delta_r = \frac{B_{\text{max}} - B_{\text{min}}}{B_{\text{max}} + B_{\text{min}}}$, JET 1975: $\varepsilon_r = 2\delta_r$

Шпаргалка — числа прикладів

	ДІІІ-D #203702, кадр 75	JET 1975 (модель)
вісь (R_{ax}, Z_{ax}) , м	(1,678; 0,026)	(3,183; 0)
R_{geo} , a , A	1,656 м; 0,605 м; 2,74	2,96 м; 1,25 м; 2,37
κ ; δ_u/δ_l	1,70; 0,38/0,69	1,68; 0,348/0,348
S , V	1,77 м ² ; 17,7 м ³	8,12 м ² ; 145,5 м ³
$ I_p $; B_φ на осі	1,36 МА; 1,93 Тл	3,80 МА; 2,77 Тл (вакуум, R_0)
q_0 ; q_{95}	0,68–0,71; 3,70	2,80; 5,51
Х-точка	(1,241; –1,096) м, $\psi_N = 1$	немає (лімітер)

Бібліографія

- [1] Фізика плазми всередині токамака : навчальний посібник (методичка) / `manual/main.pdf`. – 2026. Розд. 3 (рівновага), 4 (геометричні ефекти), 5 (анатомія токамака: JET).
- [2] Путівник гіпотезами проекту / `guide/main.pdf`. – 2026.
- [3] Реєстр встановленого (E1–E35, R1–R9) / `Our try/00-decisions/ESTABLISHED.md`. – Ред. 2026-09-29.
- [4] Реєстр візуалізації (VE, VR, VC, VQ) / `tokamak-3d-viz/findings.md`. – Ред. 2026-09-29.
- [5] tokamak-3d-viz : візуалізація рівноваги DIII-D #203702 (кадр 75, $t = 1760$ мс), трасування силових ліній, острови, перерізи Пуанкаре : програмний код (MIT) / тека `tokamak-3d-viz`; опис конвенцій — `docs/PHYSICS.md`. – 2026. Дані рівноваги: Fusion Equilibrium Challenge (Sophelio / General Atomics), CC BY 4.0.
- [6] tokamak-3d-viz : 3D-реконструкція JET (проект 1975 і стан 1983) — база розмірів `machines/jet1975`, геометричне ядро, рівновага Соловйова, гофрування : програмний код / тека `tokamak-3d-viz`; опис — `docs/JET-EQUILIBRIUM.md`. – 2026.
- [7] Nakkina et al. The Fusion Equilibrium Challenge: Inferring Magnetic Geometry Without Magnetic Diagnostics // `arXiv:2609.01750`. – 2026. Набір даних `Sophelio/fusion-equilibrium-challenge`, CC BY 4.0.
- [8] The JET Project : Design Proposal for the Joint European Torus : EUR 5516e (EUR-JET-R5). – Luxembourg : Commission of the European Communities, 1976. – 678 p. – © CECA, CEE, CEEA.
- [9] Cerfon A. J., Freidberg J. P. «One size fits all» analytic solutions to the Grad-Shafranov equation // *Physics of Plasmas*. – 2010. – Vol. 17, No. 3. – 032502. – DOI: 10.1063/1.3328818. (Поза корпусом; використано лише як метод у коді проекту.)
- [10] Miller R. L., Chu M. S., Greene J. M., Lin-Liu Y. R., Waltz R. E. Noncircular, finite aspect ratio, local equilibrium model // *Physics of Plasmas*. – 1998. – Vol. 5, No. 4. – P. 973–978. (Поза корпусом; джерело параметризації D-перерізу.)